

controle-reembolso-fullstack

Documentação Técnica Completa do Projeto

Backend · Node.js · Express · Prisma · TypeScript · Jest

Maio / 2026

Este documento explica em detalhe tudo que existe dentro do projeto: as ferramentas utilizadas, a estrutura de pastas (o que cada arquivo faz), o fluxo completo de uma requisição HTTP, a modelagem do banco de dados, as regras de negócio, as validações e os testes automatizados. O objetivo é que você consiga se virar dentro do projeto com autonomia.

1. Ferramentas Utilizadas

O projeto é um backend escrito em **TypeScript** rodando sobre **Node.js**. Cada ferramenta foi escolhida com um propósito específico:

Ferramenta / Biblioteca	Para que serve no projeto
Node.js	Ambiente de execução JavaScript no servidor. É o motor que roda o código.
Express 5	Framework HTTP. Cria o servidor, define rotas, recebe requisições e envia respostas.
TypeScript	Superset do JS que adiciona tipagem estática. Previne bugs em tempo de compilação e melhora a legibilidade.
Prisma (ORM)	Faz a comunicação com o banco de dados. Permite escrever queries em TypeScript ao invés de SQL puro. Também gera as migrations.
SQLite	Banco de dados relacional em arquivo único (.db). Usado por praticidade no ambiente de desenvolvimento e testes — zero configuração.
JWT (jsonwebtoken)	Gera e verifica tokens de autenticação. O usuário faz login, recebe um token, e usa esse token em todas as requisições seguintes.
bcrypt	Criptografa senhas antes de salvar no banco. Nunca armazenamos a senha em texto puro.
Zod	Biblioteca de validação de esquemas. Valida que os dados recebidos no body/params/query têm o formato correto antes de chegar na lógica de negócio.
dayjs	Biblioteca para manipulação de datas. Formata e converte datas no padrão brasileiro (DD/MM/YYYY).
cors	Middleware que permite que o frontend (rodando em outra porta) acesse a API.
Jest	Framework de testes. Roda os arquivos .test.ts e reporta os resultados.
Supertest	Simula requisições HTTP reais para a API durante os testes — sem precisar subir o servidor.
ts-jest	Plugin que permite ao Jest entender e executar arquivos TypeScript diretamente.
tsx	Executa arquivos TypeScript em desenvolvimento sem precisar compilar antes (substitui ts-node).
dotenv	Carrega variáveis de ambiente do arquivo .env (ex: JWT_SECRET, DATABASE_URL).

Scripts disponíveis no package.json:

Comando	O que faz
npm run dev	Sobe o servidor em modo de desenvolvimento usando tsx (hot-reload manual)
npm test	Roda todos os arquivos *.test.ts com Jest em modo serial (--runInBand) para evitar conflitos no banco

2. Estrutura de Pastas — O que Cada Arquivo Faz

O projeto tem apenas o backend implementado. A pasta raiz contém a pasta **backend/** (todo o código-fonte) e a pasta **docs/** (documentação em markdown). Abaixo está a árvore completa com explicação de cada item:

Caminho	Descrição
controle-reembolso-fullstack/	Raiz do repositório
backend/	Toda a aplicação Node.js
backend/package.json	Dependências, scripts (dev, test) e metadados do projeto
backend/tsconfig.json	Configurações do compilador TypeScript
backend/jest.config.js	Diz ao Jest: usar ts-jest, buscar arquivos *.test.ts, rodar em ambiente Node
backend/prisma.config.ts	Configuração do Prisma CLI (aponta para o schema)
backend/prisma/	Tudo relacionado ao banco de dados
backend/prisma/schema.prisma	Define os modelos (tabelas), enums, relacionamentos e o provider (SQLite)
backend/src/	Código-fonte principal da aplicação
backend/src/server.ts	Ponto de entrada: importa app.ts e sobe o servidor na porta 3000
backend/src/app.ts	Configura o Express: registra middlewares globais (cors, json) e monta as rotas
backend/src/@types/express.d.ts	Estende a tipagem do Express para incluir req.user (o usuário autenticado)
backend/src/errors/AppError.ts	Classe customizada de erro com statusCode. Lançada nos controllers para erros esperados
backend/src/lib/prisma.ts	Cria e exporta a instância única do PrismaClient (conexão com o banco)
backend/src/lib/formatDate.ts	Funções utilitárias de formatação de data usando dayjs (formatDate, formatDateTime, parseDate)
backend/src/schemas/	Schemas Zod globais reutilizáveis
backend/src/schemas/params.schema.ts	Valida que o :id na URL é um UUID válido
backend/src/schemas/query.schema.ts	Valida query params da listagem (ex: ?status=ENVIADO)
backend/src/middlewares/	Funções que rodam ANTES do controller em cada requisição
backend/src/middlewares/auth.middleware.ts	Verifica e decodifica o JWT do cabeçalho Authorization
backend/src/middlewares/role.middleware.ts	Verifica se o papel (role) do usuário é permitido para aquela rota
backend/src/middlewares/error.middleware.ts	Captura erros lançados nos controllers e retorna o HTTP correto
backend/src/middlewares/validate-params.middleware.ts	Valida req.params com um schema Zod (ex: UUID no :id)
backend/src/middlewares/validate-query.middleware.ts	Valida req.query com um schema Zod
backend/src/routes/	Define quais middlewares e controllers atendem cada endpoint

backend/src/routes/auth.routes.ts	Rotas: POST /auth/login, GET /auth/me
backend/src/routes/user.routes.ts	Rotas: POST /users, GET /users (apenas ADMIN)
backend/src/routes/category.routes.ts	Rotas: GET /categories, POST /categories, PUT /categories/:id
backend/src/routes/reimbursement.routes.ts	Rotas de todas as ações de reembolso (criar, listar, enviar, aprovar, etc.)
backend/src/modules/	Código organizado por funcionalidade (módulo)
backend/src/modules/auth/controllers/login.controller.ts	Lógica de login: valida credenciais, gera JWT
backend/src/modules/auth/schemas/login.schema.ts	Schema Zod para validar email e senha do login
backend/src/modules/user/controllers/create-user.controller.ts	Cria novo usuário com senha criptografada
backend/src/modules/user/controllers/list-users.controller.ts	Lista todos os usuários (somente ADMIN)
backend/src/modules/user/schemas/create-user.schema.ts	Valida dados de criação de usuário
backend/src/modules/category/controllers/create.controller.ts	Cria nova categoria
backend/src/modules/category/controllers/list.controller.ts	Lista categorias
backend/src/modules/category/controllers/update.controller.ts	Atualiza nome e status da categoria
backend/src/modules/category/schemas/category.schema.ts	Schema Zod para criar/atualizar categoria
backend/src/modules/reimbursement/controllers/	Um arquivo por ação de reembolso
create.controller.ts	Cria solicitação com status RASCUNHO
list.controller.ts	Lista solicitações filtradas por role do usuário
show.controller.ts	Detalha uma solicitação específica (com histórico e anexos)
submit.controller.ts	Envia solicitação de RASCUNHO para ENVIADO
update.controller.ts	Edita solicitação (só se estiver em RASCUNHO)
approve.controller.ts	Aprova solicitação ENVIADA (somente GESTOR)
reject.controller.ts	Rejeita solicitação ENVIADA com justificativa (somente GESTOR)
pay.controller.ts	Marca solicitação APROVADA como PAGA (somente FINANCEIRO)
cancel.controller.ts	Cancela solicitação em RASCUNHO ou ENVIADO (somente dono)
history.controller.ts	Lista o histórico de ações de uma solicitação
backend/src/modules/attachment/controllers/	
create-attachment.controller.ts	Adiciona um anexo à solicitação
list-attachments.controller.ts	Lista os anexos de uma solicitação
backend/src/tests/	Testes de integração

<code>auth.test.ts</code>	Testes de autenticação (login válido, senha errada)
<code>user.test.ts</code>	Testes de criação e listagem de usuários
<code>category.test.ts</code>	Testes de CRUD de categorias e controle de acesso
<code>reimbursement.test.ts</code>	Testes do fluxo completo de reembolso
<code>docs/</code>	Documentação em markdown do projeto

3. Fluxo Completo de uma Requisição HTTP

Toda requisição que chega na API percorre exatamente o mesmo caminho, em camadas. Entender esse caminho é entender como o projeto inteiro funciona.

1. Cliente (Postman/Frontend)	Envia a requisição HTTP com headers, body e token
2. CORS Middleware	Permite que origens externas acessem a API
3. express.json()	Converte o body da requisição de JSON para objeto JS
4. Rota (routes/)	Identifica qual endpoint foi chamado e quais middlewares aplicar
5. authMiddleware	Extrai o token JWT do header, verifica assinatura, busca o usuário no banco e coloca em req.user
6. validateParams / validateQuery	Zod valida se :id é UUID válido ou se query params têm formato correto
7. roleMiddleware	Verifica se req.user.role está na lista de roles permitidas para a rota
8. Controller	Recebe req/res, executa a lógica de negócio, faz queries via Prisma e retorna o JSON de resposta
9. errorMiddleware	Se qualquer camada anterior lançou um erro (AppError ou ZodError), captura e retorna o HTTP correto

Exemplo Concreto: POST /reimbursements/:id/approve

Vamos seguir passo a passo o que acontece quando um Gestor aprova uma solicitação:

Passo 1	Cliente envia POST /reimbursements/abc-123/approve Authorization: Bearer eyJhbGci...
Passo 2	Rota identifica o endpoint Em reimbursement.routes.ts: router.post('/:id/approve', authMiddleware, validateParams(idParamsSchema), roleMiddleware([GESTOR]), approveRequestController)
Passo 3	authMiddleware executa Extrai 'eyJhbGci...' do header. Chama jwt.verify() com JWT_SECRET. Decodifica: { userId: 'xyz', role: 'GESTOR' }. Busca o usuário no banco via prisma.user.findUnique. Coloca em req.user e chama next().
Passo 4	validateParams executa Chama idParamsSchema.parse({ id: 'abc-123' }). Zod verifica se 'abc-123' é um UUID válido. Se não for, lança ZodError que vai parar no errorMiddleware. Se for válido, chama next().

Passo 5	roleMiddleware([GESTOR]) executa Verifica se req.user.role === 'GESTOR'. Se não for, retorna 403 { message: 'Usuário sem permissão' }. Se for, chama next().
Passo 6	approveRequestController executa Busca a solicitação pelo id no banco. Se não encontrada: lança <code>AppError(404)</code> . Se status !== ENVIADO: lança <code>AppError(400)</code> . Atualiza status para APROVADO. Cria registro no histórico (action: APPROVED). Retorna 200 com a solicitação atualizada.
Passo 7	Resposta retorna ao cliente HTTP 200 + JSON da solicitação com status: 'APROVADO'

Todos os Endpoints Disponíveis

Método	Endpoint	Roles Permitidos	Descrição
POST	/auth/login	Público	Autenticação — retorna JWT
GET	/auth/me	Autenticado	Retorna dados do usuário logado
POST	/users	ADMIN	Cria novo usuário
GET	/users	ADMIN	Lista todos os usuários
GET	/categories	Autenticado	Lista categorias
POST	/categories	ADMIN	Cria categoria
PUT	/categories/:id	ADMIN	Atualiza categoria
POST	/reimbursements	COLABORADOR	Cria solicitação (status: RASCUNHO)
GET	/reimbursements	Autenticado	Lista solicitações (filtrada por role)
GET	/reimbursements/:id	Autenticado	Detalha solicitação
PUT	/reimbursements/:id	COLABORADOR	Edita solicitação (só RASCUNHO)
POST	/reimbursements/:id/submit	COLABORADOR	Envia para análise (RASCUNHO→ENVIADO)
POST	/reimbursements/:id/approve	GESTOR	Aprova (ENVIADO→APROVADO)
POST	/reimbursements/:id/reject	GESTOR	Rejeita com justificativa (ENVIADO→REJEITADO)
POST	/reimbursements/:id/pay	FINANCEIRO	Marca como paga (APROVADO→PAGO)
POST	/reimbursements/:id/cancel	COLABORADOR (dono)	Cancela (RASCUNHO ou ENVIADO→CANCELADO)
GET	/reimbursements/:id/history	Autenticado	Lista histórico de ações
POST	/reimbursements/:id/attachments	Autenticado	Adiciona anexo

GET	/reimbursements/:id/attachments	Autenticado	Lista anexos
-----	---------------------------------	-------------	--------------

4. Modelagem do Banco de Dados

O banco é SQLite (arquivo local) e é gerenciado pelo Prisma. O arquivo `prisma/schema.prisma` é a fonte de verdade: define os modelos, tipos e relacionamentos. O Prisma lê esse arquivo e gera as migrations SQL automaticamente.

4.1 Enums — Valores Fixos

Enums são tipos que só aceitam um conjunto fechado de valores. Eles evitam strings mágicas soltas no código e garantem consistência no banco.

UserRole	RequestStatus	RequestAction
COLABORADOR GESTOR FINANCEIRO ADMIN	RASCUNHO ENVIADO APROVADO REJEITADO PAGO CANCELADO	CREATED UPDATED SUBMITTED APPROVED REJECTED PAID CANCELED
Controla permissões (RBAC) — define o que cada usuário pode fazer no sistema	Estado atual da solicitação — fluxo que avança conforme ações	Tipo de ação registrado no histórico para rastreabilidade

4.2 As 5 Tabelas do Banco

Tabela: users		
Campo	Tipo	Descrição
id	UUID	Chave primária gerada automaticamente
name	String	Nome completo do usuário
email	String UNIQUE	Email — não pode repetir no banco
password	String	Hash bcrypt da senha — NUNCA texto puro
role	UserRole (enum)	Perfil do usuário — define suas permissões
createdAt	DateTime	Preenchido automaticamente na criação
updatedAt	DateTime	Atualizado automaticamente em toda alteração

Tabela: categories		
Campo	Tipo	Descrição
id	UUID	Chave primária
name	String	Nome da categoria (ex: Transporte, Alimentação)
active	Boolean (default true)	Soft delete — categorias inativas não somem, apenas ficam bloqueadas para novas solicitações
createdAt	DateTime	Preenchido automaticamente
updatedAt	DateTime	Atualizado automaticamente

Tabela: reimbursement_requests		
Campo	Tipo	Descrição
id	UUID	Chave primária
userId	UUID (FK users)	Quem criou a solicitação
categoryId	UUID (FK categories)	Qual categoria foi usada
description	String	Descrição da despesa (mínimo 3 caracteres)
amount	Decimal	Valor da despesa — deve ser positivo
expenseDate	DateTime	Data em que a despesa ocorreu
status	RequestStatus (enum)	Estado atual (começa como RASCUNHO)
rejectionReason	String? (nullable)	Obrigatório apenas quando status = REJEITADO
createdAt	DateTime	Preenchido automaticamente
updatedAt	DateTime	Atualizado automaticamente

Tabela: attachments		
Campo	Tipo	Descrição
id	UUID	Chave primária
requestId	UUID (FK reimbursement_requests)	A qual solicitação o anexo pertence
fileName	String	Nome original do arquivo
fileType	String	Tipo MIME (ex: image/jpeg, application/pdf)
fileUrl	String	URL simulada do arquivo (sem upload real neste projeto)
createdAt	DateTime	Preenchido automaticamente

Tabela: request_histories		
Campo	Tipo	Descrição
id	UUID	Chave primária
requestId	UUID (FK reimbursement_requests)	A qual solicitação pertence
userId	UUID (FK users)	Quem executou a ação
action	RequestAction (enum)	Tipo da ação (CREATED, APPROVED, etc.)
observation	String? (nullable)	Mensagem adicional (ex: motivo da rejeição)
createdAt	DateTime	Quando a ação ocorreu

4.3 Relacionamentos entre Tabelas

Os relacionamentos definem como as tabelas se conectam através das chaves estrangeiras (FK):

('Relacionamento', 'Cardinalidade', 'Explicação')	
User → ReimbursementRequest	[1 para N, 'Um usuário pode ter MUITAS solicitações, mas cada solicitação pertence a UM usuário (campo userId)']
Category → ReimbursementRequest	[1 para N, 'Uma categoria pode estar em MUITAS solicitações, mas cada solicitação tem UMA categoria (campo categoryId)']
ReimbursementRequest → Attachment	[1 para N, 'Uma solicitação pode ter MUITOS anexos, mas cada anexo pertence a UMA solicitação (campo requestId)']
ReimbursementRequest → RequestHistory	[1 para N, 'Uma solicitação pode ter MUITOS registros de histórico (um por ação)']
User → RequestHistory	[1 para N, 'Um usuário pode ter executado MUITAS ações registradas no histórico']

Relacionamento → Explicação	Cardinalidade
User → ReimbursementRequest — Um usuário pode ter MUITAS solicitações, mas cada solicitação pertence a UM usuário (campo userId)	1 para N
Category → ReimbursementRequest — Uma categoria pode estar em MUITAS solicitações, mas cada solicitação tem UMA categoria (campo categoryId)	1 para N
ReimbursementRequest → Attachment — Uma solicitação pode ter MUITOS anexos, mas cada anexo pertence a UMA solicitação (campo requestId)	1 para N
ReimbursementRequest → RequestHistory — Uma solicitação pode ter MUITOS registros de histórico (um por ação)	1 para N
User → RequestHistory — Um usuário pode ter executado MUITAS ações registradas no histórico	1 para N

5. Regras de Negócio

Regras de negócio são as restrições e comportamentos que o sistema deve garantir, independente de quem faz a requisição. Elas são implementadas diretamente nos controllers.

5.1 Fluxo de Status — Máquina de Estados

Status	Quem pode chegar aqui	O que pode acontecer
RASCUNHO	Colaborador — ao criar a solicitação	Editar, Enviar (→ENVIADO), Cancelar (→CANCELADO)
ENVIADO	Colaborador — ao chamar POST /submit	Aprovar (→APROVADO), Rejeitar (→REJEITADO), Cancelar (→CANCELADO)
APROVADO	Gestor — ao chamar POST /approve	Pagar (→PAGO)
REJEITADO	Gestor — ao chamar POST /reject	Nenhuma — estado terminal
PAGO	Financeiro — ao chamar POST /pay	Nenhuma — estado terminal
CANCELADO	Colaborador — ao chamar POST /cancel	Nenhuma — estado terminal

5.2 Permissões por Perfil (RBAC)

RBAC = Role-Based Access Control. O sistema tem 4 perfis, cada um com ações específicas permitidas:

COLABORADOR

- Criar solicitação (inicia como RASCUNHO)
- Editar solicitação PRÓPRIA (somente em RASCUNHO)
- Enviar solicitação para análise (RASCUNHO → ENVIADO)
- Cancelar solicitação PRÓPRIA (RASCUNHO ou ENVIADO → CANCELADO)
- Visualizar APENAS suas próprias solicitações
- Ver histórico das suas solicitações
- Adicionar/listar anexos nas suas solicitações

GESTOR

- Visualizar solicitações com status ENVIADO
- Aprovar solicitações (ENVIADO → APROVADO)
- Rejeitar solicitações com justificativa obrigatória (ENVIADO → REJEITADO)
- Ver histórico de solicitações ENVIADAS
- NÃO pode criar, editar ou cancelar solicitações

FINANCEIRO

- Visualizar solicitações com status APROVADO
- Marcar solicitações como pagas (APROVADO → PAGO)
- Ver histórico de solicitações APROVADAS ou PAGAS
- NÃO pode aprovar, rejeitar ou criar solicitações

ADMIN

- Criar e listar usuários
- Criar, listar e atualizar categorias
- Ver todas as solicitações (sem filtro de status)
- Acesso completo ao sistema

5.3 Regras de Negócio Implementadas no Código

Criação de Solicitação

- Somente COLABORADOR pode criar (roleMiddleware)
- A categoria informada deve existir e estar active = true
- description deve ter mínimo 3 caracteres (Zod)
- amount deve ser um número positivo (Zod)
- expenseDate é convertida via `dayjs.parseDate()` para garantir formato válido
- Status inicial = RASCUNHO (definido no schema do Prisma como default)
- Histórico é criado automaticamente com action: CREATED

Edição de Solicitação

- Somente o dono da solicitação pode editar (`request.userId !== user.id` → erro 403)
- Somente solicitações em RASCUNHO podem ser editadas (`status !== RASCUNHO` → erro 400)
- O campo status NÃO está no `updateRequestSchema` — usuário não pode alterar status manualmente
- Histórico é criado com action: UPDATED

Envio para Análise

- Somente o dono pode enviar (verifica `request.userId === user.id`)
- Somente RASCUNHO pode ir para ENVIADO
- Histórico é criado com action: SUBMITTED

Aprovação

- Somente GESTOR pode aprovar (roleMiddleware)
- Somente ENVIADO pode ser aprovado (verifica status)
- Histórico é criado com action: APPROVED

Rejeição

- Somente GESTOR pode rejeitar (roleMiddleware)
- Somente ENVIADO pode ser rejeitado
- O campo reason é obrigatório e deve ter mínimo 5 caracteres (rejectRequestSchema)
- O motivo é salvo em rejectionReason na tabela reimbursement_requests
- Histórico é criado com action: REJECTED e observation = motivo

Pagamento

- Somente FINANCEIRO pode pagar (roleMiddleware)
- Somente APROVADO pode ser pago
- Histórico é criado com action: PAID

Cancelamento

- Somente o dono pode cancelar (verifica request.userId === user.id)
- Somente RASCUNHO ou ENVIADO podem ser cancelados
- Histórico é criado com action: CANCELED

Visualização (listagem e detalhe)

- COLABORADOR: vê apenas suas próprias solicitações (where: { userId: user.id })
- GESTOR: vê apenas solicitações com status ENVIADO
- FINANCEIRO: vê apenas solicitações com status APROVADO
- ADMIN: vê todas (nenhum filtro aplicado)
- No detalhe (show), a mesma lógica de permissão é aplicada antes de retornar os dados

6. Validações com Zod

Zod é a biblioteca de validação de dados. Ela garante que o que chega nos controllers tem o formato correto. Cada schema define um 'contrato' de entrada.

Como o Zod funciona no projeto

Quando você chama `schema.parse(req.body)`, o Zod verifica cada campo. Se qualquer campo for inválido, ele lança um `ZodError`. O `errorMiddleware` captura esse erro e retorna HTTP 400 com a mensagem do primeiro problema encontrado.

loginSchema modules/auth/schemas/login.schema.ts		
Campo	Validação Zod	O que verifica
email	<code>string().email()</code>	Deve ser um email válido (ex: user@email.com)
password	<code>string().min(6)</code>	Deve ter no mínimo 6 caracteres

createRequestSchema modules/reimbursement/schemas/create.schema.ts		
Campo	Validação Zod	O que verifica
categoryId	<code>string()</code>	ID da categoria — validação de existência é feita no banco
description	<code>string().min(3)</code>	Descrição deve ter no mínimo 3 caracteres
amount	<code>number().positive()</code>	Valor deve ser um número positivo (maior que 0)
expenseDate	<code>string()</code>	Data no formato string (processada depois pelo dayjs)

updateRequestSchema modules/reimbursement/schemas/update.schema.ts		
Campo	Validação Zod	O que verifica
categoryId	<code>string()</code>	Mesmo que createRequest — status NÃO está aqui
description	<code>string().min(3)</code>	—
amount	<code>number().positive()</code>	—
expenseDate	<code>string()</code>	—

rejectRequestSchema modules/reimbursement/schemas/reject.schema.ts		
Campo	Validação Zod	O que verifica
reason	<code>string().min(5)</code>	Justificativa de rejeição — obrigatória e com mínimo 5 chars

idParamsSchema schemas/params.schema.ts		
Campo	Validação Zod	O que verifica
id	<code>string().uuid()</code>	O :id da URL deve ser um UUID válido — evita queries inválidas no banco

listRequestsQuerySchema schemas/query.schema.ts		
Campo	Validação Zod	O que verifica
status	<code>nativeEnum(RequestStatus).optional()</code>	Query param opcional ?status=ENVIADO — só aceita valores do enum

loginSchema (create user) modules/user/schemas/create-user.schema.ts		
Campo	Validação Zod	O que verifica
name	<code>string()</code>	Nome do usuário
email	<code>string().email()</code>	Email válido
password	<code>string().min(6)</code>	Senha com mínimo 6 caracteres
role	<code>nativeEnum(UserRole)</code>	Perfil — deve ser um dos valores do enum UserRole

7. Testes Automatizados

O projeto usa **Jest** + **Supertest** para testes de integração. Cada teste faz requisições HTTP reais à API e verifica os resultados. Os testes usam o banco SQLite real (não um mock), garantindo que a lógica de banco também está sendo testada.

7.1 Como os Testes Funcionam — Estrutura

Conceito	Explicação
<code>describe()</code>	Agrupa testes relacionados. Cada arquivo tem um <code>describe</code> principal (ex: 'Auth', 'Reimbursements')
<code>it() / test()</code>	Define um caso de teste individual. O texto descreve o comportamento esperado
<code>beforeEach()</code>	Executa ANTES de cada teste. Limpa o banco e recria os dados necessários — garante isolamento entre testes
<code>afterAll()</code>	Executa DEPOIS de todos os testes do arquivo. Desconecta o Prisma do banco
<code>expect()</code>	Faz asserções — verifica se o resultado é o esperado
<code>supertest</code>	Cria um cliente HTTP que faz requisições ao app Express sem precisar subir o servidor
<code>--runInBand</code>	Flag do Jest que roda os testes em série (um de cada vez), evitando conflitos no banco compartilhado

7.2 Padrão de cada teste — o ciclo completo

Cada teste segue exatamente este padrão:

1. ARRANGE (preparar)	<code>beforeEach</code> limpa o banco inteiro e cria os usuários e dados necessários para aquele grupo de testes. Tokens JWT são obtidos fazendo login real.
2. ACT (agir)	O teste faz uma requisição HTTP usando <code>request(app).get/post/put()</code> com headers e body.
3. ASSERT (verificar)	<code>expect()</code> verifica status HTTP, campos do body, valores específicos — confirmando que a regra de negócio funcionou.

7.3 Os 4 Arquivos de Teste

auth.test.ts 2 testes	
Nome do Teste	O que faz / O que verifica
✓ deve autenticar com credenciais válidas	Cria admin no banco, faz POST /auth/login com email e senha corretos. Espera: status 200, body com token e user.email correto.
✓ não deve autenticar com senha inválida	Cria admin, faz POST /auth/login com senha errada. Espera: status 401.

user.test.ts 3 testes	
Nome do Teste	O que faz / O que verifica
✓ deve criar um novo usuário	ADMIN faz POST /users com name, email, password, role: COLABORADOR. Espera: status 201, body.email correto.
✓ deve listar usuários (ADMIN)	ADMIN faz GET /users. Espera: status 200, array com pelo menos 1 usuário.
✓ não deve permitir acesso sem token	GET /users sem Authorization header. Espera: status 401.

category.test.ts 5 testes	
Nome do Teste	O que faz / O que verifica
✓ deve listar categorias	Cria categoria no banco, ADMIN faz GET /categories. Espera: status 200, array com pelo menos 1 item.
✓ admin deve criar uma categoria	ADMIN faz POST /categories com { name: 'Alimentação' }. Espera: status 201, body.name correto, body.active = true.
✓ colaborador não deve criar categoria	COLABORADOR faz POST /categories. Espera: status 403 (sem permissão).
✓ admin deve atualizar uma categoria	ADMIN faz PUT /categories/:id com novo nome e active: false. Espera: status 200, dados atualizados.
✓ não deve listar sem autenticação	GET /categories sem token. Espera: status 401.

reimbursement.test.ts		9 testes
Nome do Teste	O que faz / O que verifica	
✓ deve criar solicitação (RASCUNHO)	COLABORADOR faz POST /reimbursements. Espera: status 201, body.status = 'RASCUNHO'.	
✓ colaborador vê apenas suas solicitações	COLABORADOR cria 1 solicitação, faz GET /reimbursements. Espera: status 200, array com exatamente 1 item.	
✓ deve enviar solicitação	COLABORADOR cria e faz POST /reimbursements/:id/submit. Espera: status 200, body.status = 'ENVIADO'.	
✓ não deve enviar duas vezes	Envia a mesma solicitação duas vezes. Espera: segunda chamada retorna status 400.	
✓ gestor aprova solicitação	COLABORADOR cria e envia, GESTOR faz POST /approve. Espera: status 200, body.status = 'APROVADO'.	
✓ colaborador não pode aprovar	COLABORADOR tenta fazer POST /approve na própria solicitação. Espera: status 403.	
✓ gestor rejeita com justificativa	COLABORADOR cria e envia, GESTOR faz POST /reject com { reason: 'Inconsistente' }. Espera: status 200, body.status = 'REJEITADO'.	
✓ financeiro paga solicitação aprovada	Fluxo completo: criar → enviar → aprovar → FINANCEIRO faz POST /pay. Espera: status 200, body.status = 'PAGO'.	
✓ deve listar histórico	COLABORADOR cria e envia, faz GET /reimbursements/:id/history. Espera: status 200, array com pelo menos 1 registro.	

7.4 Por que os testes são importantes?

Isolamento garantido	O beforeEach limpa o banco antes de cada teste. Isso significa que cada teste começa do zero, sem resíduos de testes anteriores. Um teste nunca interfere no outro.
Testes de integração, não unitários	Os testes do projeto são de integração: eles testam a API completa, do HTTP até o banco. Isso inclui middlewares, validações, regras de negócio e queries. É mais confiável que testes unitários isolados.
Testes de autorização	Vários testes verificam que perfis ERRADOS não conseguem acessar endpoints (retornando 403). Isso garante que a segurança está funcionando.
Fluxo completo testado	O teste 'financeiro paga solicitação aprovada' faz o fluxo inteiro: criar → enviar → aprovar → pagar. Isso confirma que a máquina de estados está correta.
--runInBand obrigatório	Jest roda testes em paralelo por padrão. Como todos usam o mesmo arquivo de banco SQLite, isso causaria conflitos. --runInBand força execução sequencial.

8. Middlewares — Explicação Detalhada

Middlewares são funções que ficam no meio da requisição. Eles recebem (req, res, next) e decidem se deixam a requisição continuar (chamando next()) ou se retornam um erro antes de chegar no controller.

authMiddleware

auth.middleware.ts

1. Extraí o valor do header Authorization (ex: Bearer eyJhb...)
2. Divide pelo espaço e pega a segunda parte (o token em si)
3. Chama jwt.verify(token, JWT_SECRET) para decodificar e verificar assinatura
4. Com o userId do payload, busca o usuário no banco via prisma.user.findUnique
5. Coloca o usuário em req.user (extensão de tipo definida em @types/express.d.ts)
6. Se qualquer passo falhar: retorna 401
7. Se tudo ok: chama next() para continuar para o próximo middleware

roleMiddleware(roles)

role.middleware.ts

1. É uma função de ordem superior: recebe um array de roles e retorna um middleware
2. Verifica se req.user existe (proteção extra caso authMiddleware não tenha rodado)
3. Verifica se req.user.role está incluído no array de roles permitidas
4. Se não está: retorna 403 Usuário sem permissão
5. Se está: chama next()
6. Exemplo: roleMiddleware([UserRole.GESTOR]) só deixa passar usuários GESTOR

errorMiddleware

error.middleware.ts

1. É um middleware especial de 4 parâmetros: (error, req, res, next)
2. Registrado DEPOIS de todas as rotas em app.ts
3. Se error é AppError: retorna error.statusCode com error.message
4. Se error é ZodError: retorna 400 com issues[0].message (primeiro problema encontrado)
5. Qualquer outro erro: retorna 500 Erro interno do servidor
6. Centraliza o tratamento de erros — os controllers só precisam lançar o erro

validateParams(schema)

validate-params.middleware.ts

1. Recebe um schema Zod e retorna um middleware
2. Chama schema.parse(req.params) — valida os parâmetros da URL
3. Usado principalmente para validar que :id é um UUID válido
4. Se inválido: Zod lança ZodError que vai para errorMiddleware
5. Se válido: chama next()

validateQuery(schema)

`validate-query.middleware.ts`

1. Igual ao `validateParams`, mas valida `req.query`
2. Usado para validar query params como `?status=ENVIADO`
3. Garante que apenas valores válidos do enum `RequestStatus` são aceitos

9. Glossário Rápido

Conceitos importantes para entender o projeto:

Termo	Explicação
ORM	Object-Relational Mapper — permite usar objetos TypeScript para fazer queries ao banco sem escrever SQL
Migration	Arquivo que descreve mudanças no schema do banco — o Prisma gera e aplica automaticamente
JWT	JSON Web Token — string codificada que contém dados do usuário (userId, role) e uma assinatura digital
Middleware	Função que fica entre a requisição e o controller, podendo modificar req/res ou barrar a requisição
RBAC	Role-Based Access Control — controle de acesso baseado em papéis/cargos
UUID	Universally Unique Identifier — identificador único gerado automaticamente para cada registro
Soft Delete	Ao invés de deletar um registro, apenas marca como inativo (active: false) — preserva histórico
FK (Foreign Key)	Campo que referencia o id de outra tabela — cria a relação entre registros
Schema (Zod)	Objeto que define as regras de validação de um conjunto de dados
Enum	Tipo com valores fixos e pré-definidos — evita strings aleatórias no código
Controller	Função que recebe a requisição HTTP, executa a lógica e retorna a resposta
AppError	Classe customizada de erro com statusCode — lançada quando algo esperado dá errado (ex: não encontrado)
req.user	Campo adicionado ao objeto req pelo authMiddleware com os dados do usuário logado
next()	Função do Express que passa o controle para o próximo middleware/controller na fila
bcrypt	Algoritmo de hash para senhas — transforma a senha em uma string irreversível
supertest	Biblioteca que permite fazer requisições HTTP diretamente ao app Express nos testes
beforeEach	Hook do Jest que roda antes de cada teste — usado para limpar e preparar o banco
describe	Bloco do Jest que agrupa testes relacionados
it/test	Define um caso de teste individual no Jest
Decimal	Tipo Prisma para valores monetários — mais preciso que float para dinheiro